

CS F407 – Neural Models Lab: Learning, Depth, Activations, Output Layers

This is my solution to the lab [neur_models_lab.pdf](#).

File	Contents
neural_models_lab.py	All experiments (Tasks 1, 4A–4D, 5) in one script
results.txt	Full output of <code>python neural_models_lab.py</code> (all numbers below come from it)
README.md / REPORT.pdf	This report (specification, design, results, reflection answers) as Markdown and as PDF

Requirements: Python 3, PyTorch (CPU). Run it with:

```
python neural_models_lab.py
```

All runs are seeded, so the numbers are reproducible. Training uses Adam ($\text{lr} = 0.05$) on the full batch for 3000 steps.

Task 1 – Understanding the problem

Specification. Input space $\mathcal{X} = \{0,1\}^2$, output space $\mathcal{Y} = \{0,1\}$. The disagreement warning is $y = x_1 \text{ XOR } x_2$:

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

Sketch.

```
x2
1 | (0,1) y=1      (1,1) y=0
  |
0 | (0,0) y=0      (1,0) y=1
  +----- x1
      0           1
```

The two positive points sit on one diagonal and the two negative points on the other.

Why one line fails. Suppose a line $w \cdot x + b > 0$ selects exactly the positive points. Then $(0,1)$ and $(1,0)$ give $w_2 + b > 0$ and $w_1 + b > 0$. Adding these gives $w_1 + w_2 + 2b > 0$. But $(0,0)$ and $(1,1)$ need $b \leq 0$ and $w_1 +$

$w_2 + b \leq 0$, which sum to $w_1 + w_2 + 2b \leq 0$. That is a contradiction, so XOR is not linearly separable.

Prediction for affine + sigmoid. A single affine layer can classify at most 3 of the 4 points. With binary cross-entropy (BCE), the symmetry of XOR makes the best parameters $w = 0$, $b = 0$: every input gets $p = 0.5$ and the loss is $\ln 2 \approx 0.693$.

Result (confirms the prediction): the loss went from 0.7168 to **0.6931 = $\ln 2$** . The weights shrank to about 10^{-8} and every probability ended at exactly 0.5.

Think about it: XOR tests the claim that *the kind of representation matters, not the number of parameters*. No amount of affine capacity solves it. A nonlinear re-encoding of the input (the hidden layer) does.

Task 2 – Design of the agent

Model: 2 inputs $\rightarrow$ 2 hidden units (tanh; sigmoid and ReLU are compared in 4D) $\rightarrow$ 1 output logit. The output probability is $p = \sigma(\text{logit})$. The loss is BCE, implemented as `BCEWithLogitsLoss`, which is numerically stable. Training is gradient-based: backpropagation computes the gradients and Adam updates the parameters.

1. **Why the hidden nonlinearity is needed.** Stacking affine maps gives another affine map: $W_2(W_1x + b_1) + b_2 = W'x + b'$. Depth without a nonlinearity is therefore still one line, and Task 1 showed that fails. A nonlinearity lets the hidden layer bend the input space into a representation where XOR is linearly separable.
2. **Why sigmoid pairs with BCE.** The target is a single yes/no, i.e. a Bernoulli variable. σ maps a logit to a valid probability, and BCE is that Bernoulli distribution's negative log-likelihood. Together their gradient with respect to the logit is simply $p - y$. So the gradient does not vanish when σ saturates on a wrong answer, which it would with, for example, sigmoid + MSE.
3. **Validation criteria (what counts as learning):**
4. the final loss is far below $\ln 2$ (the "know nothing" loss), say < 0.01 ;
5. all 4 thresholded predictions are correct, with confident probabilities (< 0.1 or > 0.9);
6. the gradients are non-zero and correct: autograd matches a finite-difference estimate;
7. the result holds across several random seeds, not just one lucky run;
8. the hidden representation $h(x)$ actually makes the classes separable.

Think about it: the hidden units have no targets. What each one computes is decided entirely by the output loss. Backprop assigns credit by sending $\partial L / \partial h = W_2^T(p - y)$ back through the chain rule, and each hidden unit is pushed in whatever direction lowers the final loss.

Task 3 – LLM-assisted implementation

LLM used: Claude (Anthropic).

Prompt:

Generate minimal PyTorch code for the following model and dataset. Do not change the architecture or task. Dataset: the four XOR examples $X = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix}$, $y = [0, 1, 1, 0]$. Model: 2–2–1 network with a tanh hidden layer and a single output logit; use BCEWithLogitsLoss. Use PyTorch's default random initialisation, set a random seed, and train full-batch for 3000 CPU steps. After training, report the final loss, all four probabilities, the thresholded labels, and $W1.grad$ after `backward()`. Explain each test in one sentence.

Inspecting the code before running it (see `train()`):

Step	Code
forward pass	<code>logits = model(X)</code>
scalar loss	<code>loss = loss_fn(logits, targets)</code>
reverse-mode AD	<code>loss.backward()</code>
parameter update	<code>opt.step()</code>

Checks and corrections made while reviewing the generated code:

1. **Output/loss pairing.** I checked that the model ends in a plain `nn.Linear` (logits) with *no* `nn.Sigmoid`, because `BCEWithLogitsLoss` already applies σ internally. A final sigmoid would apply it twice and silently optimise the wrong objective. σ is applied only when reporting probabilities.
2. **Where the $p - y$ check runs (a correction after execution).** The first version checked $\partial L / \partial z = (p - y) / N$ *after* training. There every entry was about 10^{-5} and printed as `0.0000`, so the comparison was technically true but uninformative. I moved the check to initialisation, where the gradient entries are large enough to compare meaningfully.
3. **Structure.** Training is one reusable `train()` function with a `trace` hook, so the same loop records gradients (4B, 4D) and weight rows (4C) without duplicated code.

Beyond what the lab asks, I added a finite-difference gradient check, the per-example averaging check, and 20-seed robustness runs.

Think about it: reading the code alone can verify the architecture, loss choice, data, and the forward/backward/step order (e.g. the double-sigmoid bug). Whether it *learns*, whether the gradients are *numerically correct*, and how sensitive it is to seed and activation all require running it and measuring.

Task 4 – Results

Part A – Basic learning check (tanh, seed 0)

Initial loss **0.7152** → final loss **0.000083**.

x	p(y=1)	label	target
(0,0)	0.0001	0	0 ✓
(0,1)	0.9999	1	1 ✓
(1,0)	0.9999	1	1 ✓
(1,1)	0.0001	0	0 ✓

All four are correct. The learned hidden code $h(x)$ is roughly $(-1, -1)$, $(+1, -1)$, $(-1, +1)$, $(+1, +1)$ for the four inputs. Hidden unit 1 fires only for (0,1), and unit 2 only for (1,0). In h -space the two positive points are separated from the two negatives by a single line, so the network has *learned a representation that makes XOR linearly separable*.

Part B – Backpropagation check

`w1.grad` holds $\partial L / \partial W^{(1)}$. Entry (i, j) is how much the loss changes per unit change of the weight from input j to hidden unit i , evaluated at the current parameters. At initialisation (seed 0):

```
w1.grad = [[ 0.0005,  0.0006],
            [-0.0426, -0.0448]]
```

- **Mean-loss averaging:** $L = \frac{1}{4} \sum_i L_i$, and differentiation is linear, so $\partial L / \partial W = \frac{1}{4} \sum_i \partial L_i / \partial W$. Running backward on each example separately and averaging gave exactly the same tensor (max difference 0.0).
- **Finite-difference check:** central differences in float64 agree with autograd to 5.5×10^{-11} .

Part C – Symmetry experiment

Initialisation	Hidden rows of $W^{(1)}$ over training	Final loss	Learns XOR?
all weights and biases = 0	stay exactly <code>[0, 0]</code> for all 3000 steps	0.6931	No ($p = 0.5$ everywhere)
all weights and biases = 0.5	move ($0.5 \rightarrow -7.77$) but the two rows stay identical	0.4774	No (3/4; $p = 0.67$ for three inputs)

Explanation. When both hidden units have the same incoming weights, they compute the same activation $h_1 = h_2$ for every input. They also have the same outgoing weight, so they receive the same gradient $\partial L / \partial h_1 = \partial L / \partial h_2$. Identical gradients produce identical updates, so the units stay clones forever, and the network behaves like a 2–1–1 network, which cannot solve XOR.

With *zero* initialisation it is even worse. $W^{(2)} = 0$ makes $\partial L / \partial h = W^{(2)\top}(p - y) = 0$, so the first layer gets no gradient at all. Only the output bias trains, and it stays at 0 because the targets are balanced. Random initialisation is what breaks the symmetry.

Part D – Activation experiment (seed 0, gradient norm at step 0)

Hidden activation	Final loss	4/4 correct?	Early $\ \partial L / \partial W^{(1)}\ _2$
Sigmoid	0.4774	No	0.000891
Tanh	0.000083	Yes	0.061785
ReLU	0.6931	No	0.001699

Because one seed and four points prove little, I also ran 20 seeds with the same data and optimiser:

Hidden activation	Runs with 4/4 correct	Mean early $\ \partial L / \partial W^{(1)}\ _2$
Sigmoid	8 / 20	0.0072
Tanh	9 / 20	0.0329
ReLU	5 / 20	0.0373

Interpretation. With a 2-unit hidden layer every activation sometimes fails. The 2–2–1 XOR loss surface has plateaus and local minima: a run can end at loss ≈ 0.48 with 3 of 4 correct, as sigmoid did here, or stuck at $\ln 2 = 0.693$, as ReLU did here. Differences observed in this experiment:

- **Sigmoid** had the smallest early gradient (about 0.0009 for seed 0, about 0.007 on average). Its derivative is at most 0.25 (the diagnostic shows $f'(a) \approx 0.20$ – 0.25 at initialisation), and it is not zero-centred, so the backward signal through the hidden layer shrinks.
- **Tanh** had larger gradients, because its derivative is at most 1 and it is zero-centred. For seed 0 it learned the cleanest solution.
- **ReLU** failed most often. With only 2 hidden units, a unit whose pre-activation is negative for most inputs gets zero gradient on those inputs. The diagnostic shows that for seed 0 hidden unit 2 is active on only 1 of 4 inputs and unit 1 on 2 of 4. Once the units die on the inputs that matter, learning stalls at $\ln 2$.

This does not show any activation is universally best. The rankings describe only this tiny 2-unit network, this optimiser, and these seeds. A wider hidden layer, or a different learning rate, could change them.

Think about it: saturation and dead ReLUs can be told apart by logging the **pre-activations** $a^{(1)}$. A saturated sigmoid/tanh unit has $|a|$ large (e.g. $|a| > 5$), so h is near 0/1 (or ± 1) and $f'(a) = h(1-h) \approx 0$. A dead ReLU unit has $a \leq 0$, so $h = 0$ exactly and $f' = 0$ exactly. That is a hard zero, not a small number, and it holds for every input where $a \leq 0$. So: large-magnitude pre-activations mean saturation; negative pre-activations with exactly-zero outputs mean dead ReLU.

Task 5 – Three-class extension

Classes: 0 = both inactive, 1 = disagree, 2 = both active. Architecture: $2 \rightarrow 2$ (tanh) $\rightarrow$ **3 logits**, loss `CrossEntropyLoss` (softmax + negative log-likelihood). Only the output layer and loss changed.

Predictions before running (all confirmed):

1. The final weight matrix $W^{(2)}$ has shape 3×2 (outputs $\times$ hidden), plus a bias of size 3.
2. There are **3 logits** per example.
3. $\text{softmax}(z)_k = e^{\{z_k\}} / \sum_j e^{\{z_j\}}$. Every term is positive and the numerators add up to the denominator, so the probabilities sum to 1.
4. **Why $\partial L / \partial z = p - y$.** $L = -\sum_k y_k \log p_k = -z_c + \log \sum_j e^{\{z_j\}}$, where c is the true class. Differentiating gives $\partial L / \partial z_k = -y_k + e^{\{z_k\}} / \sum_j e^{\{z_j\}} = p_k - y_k$. With a mean over N examples, this is divided by N .

Numerical check of $p - y$ (at initialisation): autograd's $\partial L / \partial z$ matches $(p - y)/N$ to within 7.5×10^{-9} , i.e. float32 round-off. The first row, for $x = (0,0)$ with true class 0, is $[-0.1790, 0.0825, 0.0964]$ in both. Only the true class has a negative gradient.

Results. Loss went from 1.0591 to 0.000039, and all 4 inputs are classified correctly:

x	p(class 0)	p(class 1)	p(class 2)	predicted	target
(0,0)	0.99995	0.00005	0.00000	0	0 ✓
(0,1)	0.00002	0.99997	0.00001	1	1 ✓
(1,0)	0.00002	0.99997	0.00001	1	1 ✓
(1,1)	0.00000	0.00005	0.99995	2	2 ✓

For $x = (0,1)$ the softmax vector sums to **1.0**.

Optional diagnostic. Adding 100 to all three logits changed the probabilities by at most 2.3×10^{-10} , which is float round-off. This follows from $e^{\{z+c\}} / \sum e^{\{z+c\}} = e^{\{z\}} / \sum e^{\{z\}}$. Adding 1000, however, makes a naive `exp` overflow to `inf`, and `inf/inf = nan`. That is why stable implementations compute $\text{softmax}(z - \max z)$. The answer is mathematically the same, but the largest exponent becomes $e^0 = 1$, so nothing overflows. The max-subtracted version returned the correct probabilities.

Think about it: next-token prediction is the same problem with $K \approx 50,000$ classes. What stays the same: the softmax, the cross-entropy loss, the logit gradient $p - y$, the probabilities summing to 1, shift invariance, and the need for max-subtraction. What changes dramatically: the input is a variable-length token sequence rather than two bits. The hidden representation comes from a deep transformer with attention instead of 2 tanh units. The output matrix is $d \times 50,000$, so it dominates parameters and compute. Training needs mini-batches over huge corpora, and inference needs sampling strategies.

Reflection questions

1. **Depth vs nonlinearity.** Depth alone does nothing: stacked affine layers collapse to one affine map, which provably cannot fit XOR (Task 1: stuck at ln 2). Adding a single nonlinear hidden layer with only 2 units was enough (loss 8×10^{-5}). What matters is the *nonlinear re-representation*, not the layer count.
2. **Evidence of a useful learning signal.** Several signs together:
3. The loss fell steadily from 0.715 to 8×10^{-5} .

4. All four predictions flipped to the correct side with confidence.
5. The hidden layer developed a *new, interpretable code*: one unit per "disagree" input, which makes the classes linearly separable.
6. The gradient was *correct*, matching finite differences to 5.5×10^{-11} .

A merely non-zero gradient shows none of this. The symmetric 0.5-initialised run had non-zero gradients too, yet never solved the task.

1. **Why identical initialisation fails.** Identical hidden units compute identical outputs and so receive identical gradients and identical updates. The symmetry is never broken, and the layer acts like a single unit. With all-zero weights, $W^{(2)} = 0$ also blocks all gradient into $W^{(1)}$, so the first layer never changes at all.
2. **Effect of activation on the gradient.**
3. *Engineering observation:* at initialisation $\|\partial L / \partial W^{(1)}\|$ was about 0.0009 for sigmoid, 0.06 for tanh, and 0.0017 for ReLU (seed 0). Success rates over 20 seeds were 8, 9 and 5.
4. *Scientific explanation:* the backward pass multiplies by $f'(a)$. Sigmoid's $f' \leq 0.25$ shrinks the signal, and its outputs are not zero-centred. Tanh's $f' \leq 1$ and it is zero-centred. ReLU's f' is exactly 1 or 0, so it passes gradients undiminished through active units but blocks them completely through inactive ones. With only 2 hidden units, losing one is fatal.
5. **Why output layer and loss go together.** Together they define the probabilistic model of the target: Bernoulli $\rightarrow$ sigmoid + BCE, categorical $\rightarrow$ softmax + CE, real-valued Gaussian $\rightarrow$ linear + MSE. The matched pairs are negative log-likelihoods, so the logit gradient is simply $p - y$: well-scaled, and not killed by saturation. A mismatched pair behaves badly. Sigmoid + MSE has vanishing gradients on confident mistakes. Softmax + BCE treats mutually exclusive classes as independent.
6. **LLM: productivity vs verification.**
7. *Productivity:* the LLM produced the full training-loop boilerplate and the `BCEWithLogitsLoss / CrossEntropyLoss` wiring in seconds, and it converted the binary model to 3 classes by editing only the output layer and loss.
8. *Verification was essential:* the first $p - y$ check ran after training and printed all zeros. It "passed" without testing anything until it was moved to initialisation. Also, the single-seed run could easily have been reported as "tanh works". Only the multi-seed runs showed that the 2–2–1 network fails on over half of initialisations. Output/loss pairings (e.g. an extra sigmoid before `BCEWithLogitsLoss`) must also be checked by reading the code, because such a bug still runs without errors.
9. **Which tests scale.**
10. *Keep:* loss curves; train/validation accuracy; per-layer gradient-norm monitoring (catches vanishing, exploding, and dead units); activation/pre-activation statistics; multiple seeds; a random-initialisation check against symmetry; the softmax-sum and max-subtraction stability checks; overfitting a tiny batch as a smoke test.
11. *Too expensive:* exhaustive finite-difference checks, which need 2 forward passes per parameter (billions for an LLM). At scale these become spot-checks on a few random parameters or on a tiny

model. Exhaustive per-example gradient comparisons and printing whole weight matrices are also impractical.